

Classification Predicts Categories or Their Probabilities

Classification is supervised learning with a categorical target.



CORE CONCEPT

Logistic Regression Converts a Score into a Probability

Logistic regression first calculates a linear score: $z = \beta_0 + \beta^T x$ Then the sigmoid converts that score into a probability:

x : feature vector

β : learned coefficient vector

β_0 : intercept

z : real-valued score

e : mathematical constant approximately 2.718

$\sigma(z)$: value between 0 and 1

RUN IT AND INSPECT THE RESULT

Thresholds Turn Probabilities into Actions

```
import numpy as np

probabilities = np.array([0.90, 0.70, 0.40, 0.20])
print((probabilities >= 0.30).astype(int))
```

OUTPUT

```
[1 1 1 0]
```

Only the third decision changes.

REASONING SEQUENCE

A Confusion Matrix Counts Four Outcomes

Frame

True positive (TP):
actual positive,
predicted positive

1

Transform

False positive (FP):
actual negative,
predicted positive

2

Validate

False negative (FN):
actual positive,
predicted negative

3

Explain

True negative (TN):
actual negative,
predicted negative

4

Actual [1,1,1,0,0,0] Predicted [1,0,1,1,0,0] TP = 2 FN = 1 FP = 1 TN = 2 Total = 6.

CORE CONCEPT

Accuracy Measures Overall Correctness

$\text{accuracy} = (TP + TN) / (TP + TN + FP + FN)$ Accuracy answers: "What fraction of all predictions were correct?" It treats all rows and both error types equally. It can look strong when one class dominates, even if the minority class is never detected.

$\text{accuracy} = (TP + TN) / (TP + TN + FP + FN)$

Accuracy answers: "What fraction of all predictions were correct?"

It treats all rows and both error types equally.

It can look strong when one class dominates, even if the minority class is never detected.

Always compare with class balance and a simple baseline.

From TP 2, TN 2, FP 1, FN 1:

RUN IT AND INSPECT THE RESULT

Precision and Recall Answer Different Questions

```
from sklearn.metrics import precision_score, recall_score

print(precision_score(y_true, y_pred))
print(recall_score(y_true, y_pred))
```

OUTPUT

```
0.6666666666666666
0.6666666666666666
```

$$8 / (8 + 4) = 8/12 = 0.667$$

CORE CONCEPT

F1 Balances Precision and Recall

The F1 score is the harmonic mean of precision and recall: $F1 = 2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$. F1 is high only when both precision and recall are high. It ignores true negatives, so it should be chosen because positive-class detection matters, not simply because it is popular.

The F1 score is the harmonic mean of precision and recall:

$F1 = 2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$

F1 is high only when both precision and recall are high.

It ignores true negatives, so it should be chosen because positive-class detection matters, not simply because it is popular.

For multiclass data, macro, micro, and weighted averaging answer different questions.

Precision 0.80, recall 0.667

REASONING SEQUENCE

ROC-AUC Evaluates Ranking Across Thresholds

Frame

true positive rate,
which equals recall;

1

Transform

false positive rate FP
/ (FP + TN);

2

Validate

across many
thresholds.

3

Positive scores [0.9, 0.7] and negative scores [0.6, 0.2]. Every positive score exceeds every negative except none in this case, so all four positive-negative pairs are correctly ranked. AUC is 1.0.

RUN IT AND INSPECT THE RESULT

Multiclass Metrics Need an Averaging Rule

```
from sklearn.metrics import classification_report  
  
print(classification_report(y_true, y_pred))
```

OUTPUT

Precision, recall, F1, and support for each class plus stated averages.

This gives rare class C equal importance.

QUALITY GATE

Regression Metrics Describe Numerical Error



MAE: mean absolute error



MAE = $(1/n) \sum |y_i - \hat{y}_i|$



RMSE: square root of mean squared error;



gives larger residuals more influence.

QUALITY GATE

One Evaluation Report Needs Context



test population and time period;



class counts or target distribution;



baseline;



locked threshold;

GUIDED LAB

Guided Lab: Calculate Metrics from Counts

01

**identify the
positive class**

02

**generate probabilities
and choose a validation
threshold**

03

**calculate TP, FP,
FN, and TN by
hand**

04

**calculate accuracy,
precision, recall, and
F1**